

EVOGENUI-BENCH: Evaluating LLMs as Multi-Turn Generative UI Assistants

Yue Peng, Lanke Xia*, Zihan Wang*, Jiahao Ye*, Ke Ning, Hongyi Wen†
New York University Shanghai

Code: <https://github.com/MAPS-research/EvoGenUI-Bench>

Abstract

Large language models can generate interactive web interfaces, but reliable generative UI requires maintaining an executable artifact as user requests evolve. We introduce EVOGENUI-BENCH, a benchmark for multi-turn interface maintenance comprising 150 five-turn tasks and 750 turns across three scenarios: information presentation, executable interaction, and tool-grounded external state. We execute generated artifacts in a browser and evaluate them using screenshots, source and DOM evidence, actor traces, and runtime logs. Beyond turn-level and episode-level success, we measure cross-turn retention with Adjacent Pass Retention. Across eight models, even the strongest achieves 74.9% Turn Pass while completing only 37.3% of five-turn episodes; APR further falls to 52.4% on tool-grounded tasks. Diagnostic analysis shows that presentation failures center on information architecture, interaction failures on derived-state propagation and affordance binding, and tool-grounded failures additionally involve external-state grounding and requirement decomposition. These results reframe generative UI evaluation from judging isolated outputs to testing whether interface behavior, derived state, external state, and assistant claims remain synchronized as the artifact evolves.

1 Introduction

Large language models (LLMs) can generate task-specific interactive interfaces, including dashboards, forms, comparison views, and mini-applications. Recent work studies these interfaces as a direct response modality for LLM-based assistants (Leviathan et al., 2026; Chen et al., 2026; Zhang et al., 2026). We investigate whether LLMs can maintain an executable interface as user requirements evolve over multiple turns. Figure 1



Figure 1: **Multi-turn artifact maintenance.** Rather than generating independent pages, the model repeatedly updates a single executable interface as user requirements evolve. Each update must implement the current request, preserve still-valid requirements, and keep interface behavior, visible state, and the accompanying response consistent with the conversation history.

illustrates a conversation in which the user adds filters and views, revises constraints, and extends a workflow while expecting earlier functionality to remain intact. We refer to this setting as EVOGENUI: across turns, the model updates a single executable interface to satisfy each new request without violating requirements that remain valid.

*Equal Contribution

†Correspondence to <hongyi.wen@nyu.edu>

Single-shot evaluation does not capture the nuances in this setting.

Existing benchmarks evaluate frontend generation and revision, tool use, and agents operating within existing web environments (Zhu et al., 2025; Wu et al., 2026; Yao et al., 2025; Zhou et al., 2024). These settings largely separate artifact construction from task execution: the model either develops an interface or operates through a pre-defined environment. Generative UI collapses this distinction: the model continually revises the interface through which the task itself is performed. Reliability must therefore be assessed over the full revision sequence, including whether behavior, state, and prior requirements remain consistent as the interface evolves.

We introduce EVOGENUI-BENCH, a benchmark comprising 150 tasks and 750 turns across three scenario suites. Presentation-focused tasks evaluate the organization of dense information; interaction-focused tasks evaluate executable interfaces with local state; and tool-grounded tasks evaluate interfaces that mediate external runtime state. Each task consists of an initial interface-generation request followed by four successive revisions to the same interface.

For every requested artifact, we attempt to build and execute the generated interface in a browser. Buildable artifacts are scored for Presentation, Execution, and Alignment using the multi-surface evidence bundle defined in Section 4.1, while artifacts that cannot be built or executed are counted as failures. We measure turn-level success with Turn Pass, complete-episode success with TP@5, and cross-turn retention with Adjacent Pass Retention (APR), the share of qualifying next turns that pass given a passing previous turn. The best-performing model reaches 74.9% overall Turn Pass but completes only 37.3% of five-turn episodes. After pooling all such adjacent pairs across the eight models, tool-grounded APR is 52.4%. Failure mechanisms also differ substantially across suites.

Our contributions are: (1) EVOGENUI-BENCH, a multi-turn benchmark for evaluating the maintenance of executable interfaces across cumulative revisions; (2) a browser-based, evidence-grounded evaluation protocol that combines visual, behavioral, source-level, and runtime evidence with turn-, episode-, and transition-level reliability metrics; (3) a systematic evaluation of eight recent models spanning multiple families, revealing a substantial gap between turn-level success and sustained

reliability; and (4) a human-validated diagnostic taxonomy that identifies distinct mechanisms of interface-maintenance failure across presentation, interaction, and tool-grounded scenarios.

2 Related Work

2.1 Generative Interfaces and Iterative Development

Existing Generative UI benchmarks assess visual fidelity (Si et al., 2025), interactive behavior and executable frontend generation (Xiao et al., 2025; Zhu et al., 2025; Sun et al., 2025), and multi-file website generation (Lu et al., 2025b). MiniApp-Bench extends this line of work to principle-driven, interaction-intensive mini-applications evaluated through static inspection and agent-based exploration (Zhang et al., 2026). These benchmarks primarily focus on self-contained generation episodes rather than the evolution of a persistent interface under successive user requests.

Recent work has begun to examine generation under evolving requirements. FronTalk studies multi-turn frontend development with textual and visual feedback, identifying the forgetting or overwriting of prior features as a central challenge (Wu et al., 2026). SlopCodeBench shows that iterative code extension can satisfy intermediate checkpoints while accumulating structural degradation (Orlanski et al., 2026), and MultiChallenge documents broader failures in realistic multi-turn instruction following (Deshpande et al., 2025). EVOGENUI-BENCH extends this direction by treating the generated interface as both a persistent assistant output and the interaction layer. Each revision must incorporate the latest request while preserving prior requirements, executable behavior, and consistency with the assistant response; tool-grounded tasks additionally require synchronization with external runtime state.

2.2 Interactive Agents and Stateful Tool Use

WebArena and VisualWebArena evaluate agents operating websites, while OSWorld and Agent-Bench extend this paradigm to open-ended computer environments and broader interactive tasks (Zhou et al., 2024; Koh et al., 2024; Xie et al., 2024; Liu et al., 2024). In these benchmarks, the environment supplies the interface through which the agent acts. In EVOGENUI-BENCH, the model instead generates and continually revises that interface.

ToolLLM and the Berkeley Function Calling

Leaderboard evaluate tool selection and invocation (Qin et al., 2024; Patil et al., 2025), whereas τ -bench, ToolSandbox, and AppWorld introduce environments in which tool calls modify persistent state (Yao et al., 2025; Lu et al., 2025a; Trivedi et al., 2024). These benchmarks primarily assess whether an agent selects appropriate tools and produces the intended state changes. The tool-grounded suite of EVOGENUI-BENCH asks whether those operations are also exposed correctly through an evolving, user-operable interface, maintaining consistency among visible controls, browser interactions, local UI state, and authoritative external state.

2.3 Evaluation of Executable Interactive Artifacts

Open-ended interfaces cannot be evaluated reliably through reference renderings, static code analysis, or text-only judgments alone. SWE-bench evaluates generated code through execution in a software environment, and frontend benchmarks similarly use sandboxed builds, browser execution, and functional tests (Jimenez et al., 2024; Zhu et al., 2025). ArtifactsBench complements execution with temporal screenshots and multimodal judging to capture visual and interactive behavior (Zhang et al., 2025).

LLM-as-a-judge methods such as MT-Bench, AlpacaEval, and Prometheus provide scalable, rubric-guided evaluation of open-ended outputs while motivating careful control of evaluator evidence and bias (Zheng et al., 2023; Dubois et al., 2024; Kim et al., 2024). Executable interface maintenance requires these judgment-based methods to be grounded in execution evidence: an artifact may build successfully while exposing broken controls, retaining stale state, losing prior functionality, or contradicting the assistant response. EVOGENUI-BENCH therefore combines rendered UI observations, source and DOM evidence, browser interactions, runtime traces, textual responses, and, when applicable, authoritative external-state readback within the shared evidence bundle.

3 EVOGENUI-BENCH

3.1 Task Formulation

We formulate the evaluation of generative UIs as multi-turn artifact maintenance. Each task is a five-turn episode in which the user successively revises the same web interface rather than requesting independent pages. At each turn, the model receives the

current request, public task context, optional public tool contracts, compact evidence from earlier turns, and the most recently generated source code. It must return both a user-facing response and the complete source code for the updated interface.

The task is cumulative: a valid update must implement the new request while preserving all still-valid prior requirements. The evaluation unit is therefore an evolving executable artifact rather than an isolated page. Its behavior, visible state, and accompanying assistant response must remain mutually consistent across revisions. Section 4.1 describes how the benchmark harness executes each update and collects evaluation evidence.

3.2 Design and Construction

To prevent evaluation leakage, the benchmark separates generator-visible requests, context, and tool contracts from private validation requirements, backend state, actor-only information, and scoring criteria. Each private requirement identifies at least one observable evidence surface: the rendered UI, DOM, interaction trace, source code, assistant response, tool log, or runtime state. Validation targets the requested semantics rather than a reference implementation, allowing functionally equivalent solutions to pass while rejecting static shells and unsupported claims about interface or external state.

All tasks are human-authored to maintain a fixed five-turn revision structure, balanced coverage across scenario types, and, where applicable, deterministic external state. Machine-readable domain metadata guides task sampling and diversity auditing. The benchmark contains 150 distinct domain labels across 150 tasks, with each task pairing its domain with an interface type and a targeted challenge, such as evidence-board synchronization, state-machine simulation, linked-view propagation, or tool-backed status readback. This design reduces the risk of near-duplicate prompt templates while preserving a common evaluation structure.

A second reviewer checked each task for public-turn clarity, leakage of private validation criteria, meaningful cross-turn revision pressure, domain duplication, and requirement observability. Tasks that failed any check were revised. Appendix B.1 describes the complete construction and review workflow, and Appendix B.3 presents a tool-grounded example.

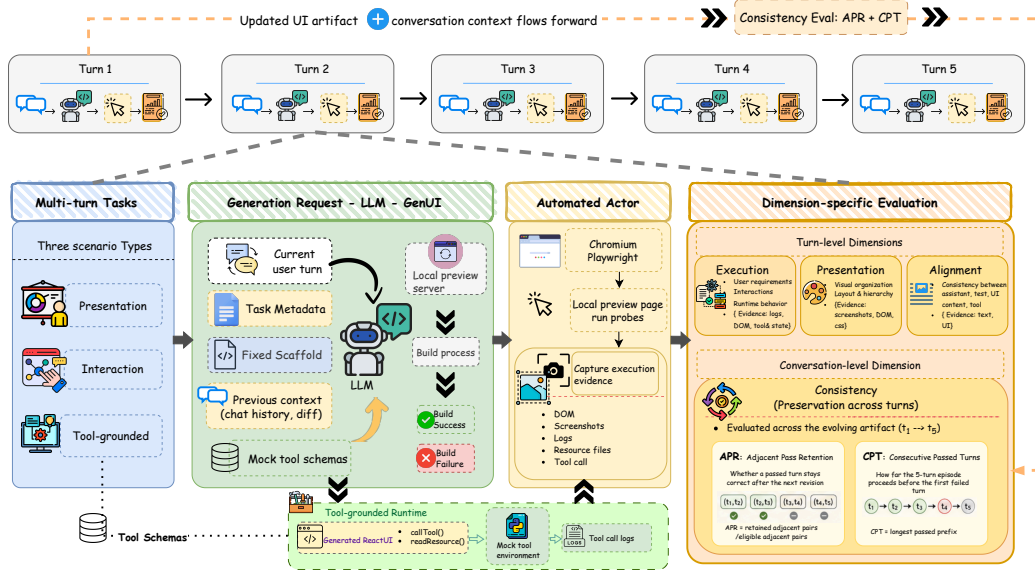


Figure 2: **Evaluation pipeline.** EVOGENUI-BENCH generates an updated interface, executes it in a browser, collects multi-surface evidence, and computes turn-level scores and derived statistical reliability metrics.

3.3 Benchmark Composition

The EVOGENUI-BENCH suite contains 150 tasks and 750 requested turns, divided equally among three 50-task scenario suites: Presentation, Interaction, and Tool-grounded. Each task comprises five user turns and a private validation contract for every turn. Presentation tasks emphasize explanatory and decision-support artifacts; Interaction tasks cover stateful mini-applications and reasoning workbenches; and Tool-grounded tasks require interfaces that read, write, reconcile, or refresh external state. The suites contain equal numbers of tasks and turns but intentionally differ in validation-contract density, as shown in Table 1.

Subset	Tasks	Domains	Tools/task	Req./turn
Presentation	50	50	0	3.0
Interaction	50	50	0	5.9
Tool-grounded	50	50	6–22	11.9
Total	150	150	–	6.9

Table 1: **Benchmark composition.** Every task contains five turns, yielding 750 requested turns in total. Req./turn denotes the mean number of private validation requirements per turn; these requirements are hidden from the generator.

4 Evidence-Grounded Evaluation

We evaluate each requested turn through interface generation, browser execution, evidence collection, and model-based scoring. Figure 2 summarizes this workflow and the evidence passed between stages.

4.1 Generation and Evidence Collection

At each turn, the benchmark harness builds the returned source code in a fixed web environment and serves the resulting interface in a browser. It records build and browser evidence, while an interaction actor operates the running interface to collect behavioral traces and observations. Execution proceeds sequentially: later turns depend on the previously generated source, collected snapshots, and, for tool-grounded tasks, restored runtime state.

If a required artifact is missing, downstream turns that depend on it are blocked rather than restarted from a clean state; the corresponding requested slots are not silently omitted from evaluation. Actor-reported interaction status is advisory rather than an official label. Instead, the recorded interaction evidence is passed to the evaluator and incorporated into the dimension-level scores described below.

4.2 Evaluation Dimensions

The model-based evaluator assigns three turn-level scores. **Presentation** evaluates the coherence, readability, and domain appropriateness of the rendered interface. **Execution** evaluates whether the current request and all relevant, still-valid prior requirements are implemented as working interface behavior or visible, evidence-supported state. **Alignment** evaluates whether the assistant response, source code, rendered UI, interaction observations, and runtime logs are mutually consistent. Appendix C.1

provides the complete scoring rubrics.

The evaluator combines three types of evidence: rendered state, comprising the final DOM and screenshot; execution evidence, comprising actor observations, tool and resource logs, and runtime snapshots; and artifact context, comprising the source summary, build status, and compact context from previous turns. Together, these inputs allow the evaluator to distinguish failures in visual organization from broken controls, stale derived state, unsupported assistant claims, and contradictions between the interface and tool-grounded state.

4.3 Reliability Metrics

We report reliability at three levels: Turn Pass (**TP**) measures correctness at an individual turn; **TP@5** and Consecutive Passed Turns (**CPT**) measure sustained success over an episode; and Adjacent Pass Retention (**APR**) measures whether success persists across adjacent revisions. All four metrics derive from the same official turn-level decision.

Formally, for turn t of episode $\mathcal{T}$, let $p_{\mathcal{T},t} \in \{0, 1\}$ denote the official pass indicator. It equals 1 only if generation succeeds, the returned source yields an evaluable artifact, and the Presentation, Execution, and Alignment scores are each at least 4. Turn Pass is the mean of these indicators over all requested model–turn slots; generation, build, and execution failures therefore remain failures under requested-slot accounting.

For an N -turn episode $\mathcal{T}$,

$$P_{\text{TP@5}}(\mathcal{T}) = \prod_{t=1}^N p_{\mathcal{T},t}, \quad \text{CPT}(\mathcal{T}) = \sum_{t=1}^N \prod_{j=1}^t p_{\mathcal{T},j}. \quad (1)$$

Because every episode contains five turns, $P_{\text{TP@5}}(\mathcal{T})$ indicates whether all five turns pass. CPT measures the length of the initial uninterrupted run of passing turns and ranges from zero to five. Suite-level TP@5 and CPT are obtained by averaging these episode-level quantities across tasks.

For comparison, we also report a diagnostic independence baseline for TP@5. For each model, let $\hat{p}_t$ denote its empirical pass rate over all requested slots at turn position t . If the five turn outcomes within an episode were independent with these position-specific pass probabilities, the expected TP@5 would be

$$P_{\text{indep}}^{\text{TP@5}} = \prod_{t=1}^5 \hat{p}_t. \quad (2)$$

Let $e_{\mathcal{T},t} = 1$ if the generation call for turn t returns a model response, and let $e_{\mathcal{T},t} = 0$ if no response is obtained because of a provider failure. Invalidly formatted responses and build failures remain countable and are assigned $p_{\mathcal{T},t} = 0$. For a set of episodes $\mathcal{S}$,

$$\text{APR}(\mathcal{S}) = \frac{\sum_{\mathcal{T} \in \mathcal{S}} \sum_{t=2}^{N_{\mathcal{T}}} e_{\mathcal{T},t} p_{\mathcal{T},t-1} p_{\mathcal{T},t}}{\sum_{\mathcal{T} \in \mathcal{S}} \sum_{t=2}^{N_{\mathcal{T}}} e_{\mathcal{T},t} p_{\mathcal{T},t-1}}. \quad (3)$$

APR therefore measures the probability that turn t passes given that turn $t - 1$ passed and the generation call for turn t returned a model response. It is an outcome-level retention measure: a failed transition may result from regression on a prior requirement, failure to satisfy the new request, or both. Appendix C.2.1 separates attributable APR failures into cases involving regression on prior requirements and cases involving only the new requirement.

TP, TP@5, and CPT are macro-averaged across tasks at the suite level, whereas APR pools all eligible adjacent transitions. Appendix C.2 describes the countable-only diagnostics and bootstrap procedure. Table 9 reports task-level confidence intervals and independence-baseline values.

4.4 Human Validation

We validate automatic pass/fail decisions on a blinded sample of 240 turn-level cases covering all eight generator models and all three suites, with 10 cases per model–suite pair and stratification by automatic pass/fail outcome. Three non-author annotators independently label every case while blinded to model identity and the automatic decision; majority vote defines the human reference. The automatic evaluator reaches 86.7% accuracy and Cohen’s $\kappa = 0.73$ against these labels, while inter-annotator agreement is Fleiss’ $\kappa = 0.78$.

5 Experiments

5.1 Models

We evaluate eight models spanning several recent model families: GPT-5.5 (OpenAI, 2026), Qwen3.6-Plus (Alibaba Cloud, 2026), Qwen3-Coder (Qwen Team, 2025), GLM-4.5-Air (GLM-4.5 Team, 2025), Gemini-3-Flash (Google DeepMind, 2025), Gemini-3.1-Pro (Google DeepMind, 2026), Claude-4.5-Haiku (Anthropic, 2025), and Claude-Opus-4.7 (Anthropic, 2026). Appendix D.1 provides the exact provider identifiers and inference settings used for each model.

Model	Presentation				Interaction				Tool-grounded				Overall			
	TP	TP@5	CPT	APR	TP	TP@5	CPT	APR	TP	TP@5	CPT	APR	TP	TP@5	CPT	APR
Claude-Opus-4.7	82.4	52.0	3.28	91.3	81.2	40.0	3.02	86.8	61.2	20.0	2.14	68.6	74.9	37.3	2.81	83.6
GPT-5.5	44.8	4.0	0.94	50.6	77.6	40.0	3.24	80.6	62.8	20.0	2.44	72.7	61.7	21.3	2.21	71.0
Claude-4.5-Haiku	76.0	32.0	3.08	82.8	56.0	8.0	1.36	60.9	21.2	0.0	0.40	20.0	51.1	13.3	1.61	65.7
Qwen3.6-Plus	60.0	12.0	1.76	72.3	46.8	10.0	1.58	64.1	25.2	0.0	0.62	34.6	44.0	7.3	1.32	61.7
Gemini-3-Flash	62.8	8.0	1.56	68.5	47.6	6.0	1.20	57.0	14.8	0.0	0.30	18.8	41.7	4.7	1.02	57.8
Gemini-3.1-Pro	38.4	8.0	1.14	70.3	25.2	12.0	0.78	79.6	7.2	0.0	0.16	29.4	23.6	6.7	0.69	68.6
Qwen3-Coder	44.0	6.0	1.44	53.5	21.2	0.0	0.72	32.0	4.4	0.0	0.14	0.0	23.2	2.0	0.77	42.2
GLM-4.5-Air	35.2	2.0	1.10	50.0	24.8	2.0	0.54	47.2	3.2	0.0	0.16	42.9	21.1	1.3	0.60	48.5
Aggregate	55.5	15.5	1.79	71.1	47.6	14.8	1.56	68.7	25.0	5.0	0.80	52.4	42.7	11.8	1.38	66.5

Table 2: **Main results.** Models are ordered by overall TP. TP is turn pass rate (%); TP@5 is the percentage of episodes in which all five turns pass; CPT is the average initial run of passed turns; and APR is adjacent pass retention (%). Overall averages the three suites for TP, TP@5, and CPT and pools eligible transitions for APR. The final row macro-averages model rows for TP, TP@5, and CPT and pools eligible transitions for APR. Bold marks column best and aggregate values.

Where required, we use provider-specific transport or output modes while preserving a common normalized output contract: non-empty assistant text accompanied by the generated source files.

5.2 Experimental Setup

Each model is evaluated on the same 150 tasks, with five requested turns per task, using a common execution harness, evidence-collection protocol, and evaluator. Execution is sequential: each turn reuses the previously generated source and collected snapshots and, for tool-grounded tasks, the restored runtime state. For the main results, we use MiMo-V2.5 (Xiaomi MiMo Team, 2026) as both the interaction actor and the evaluator. We use refreshed full-page screenshots and fixed evaluation rubrics across all evaluated models.

5.3 Main Results

Finding 1: Turn-level success overstates episode-level reliability. Claude-Opus-4.7 achieves the highest overall TP (74.9%) and APR (83.6%), yet only 37.3% of its episodes pass all five turns. Averaged across models, TP is 42.7%, whereas TP@5 is only 11.8%. This gap shows that success on individual turns does not translate directly into reliable completion of an entire revision sequence.

Observed TP@5 exceeds the independence estimate in Equation 2 for all eight models. Representative comparisons are 21.3% versus 8.7% for GPT-5.5, 7.3% versus 1.3% for Qwen3.6-Plus, 4.7% versus 1.2% for Gemini-3-Flash, and 37.3% versus 23.4% for Claude-Opus-4.7 (Table 9). Passing turns therefore cluster within episodes, potentially because episodes differ in difficulty. Accordingly, TP@5 measures end-to-end workflow success rate.

Models also exhibit distinct suite-level profiles. GPT-5.5 and Claude-Opus-4.7 both achieve 40.0% TP@5 on Interaction tasks. Qwen3.6-Plus performs substantially better on Presentation than on Tool-grounded tasks, with TP decreasing from 60.0% to 25.2%. Gemini-3.1-Pro has a low overall TP of 23.6% but relatively high APR after reaching a passing state on Presentation (70.3%) and Interaction (79.6%). This contrast illustrates the conditional nature of APR: a model may preserve a correct artifact when one is reached even if it rarely reaches such a state.

Finding 2: Tool-grounded tasks remain the most difficult after conditioning on a previous pass. Model-averaged TP decreases from 55.5% on Presentation and 47.6% on Interaction tasks to 25.0% on Tool-grounded tasks; model-averaged TP@5 similarly falls to 5.0% (Figure 3a). Tool-grounded tasks also have denser validation contracts, averaging 11.9 requirements per turn, compared with 3.0 for Presentation and 5.9 for Interaction (Table 1). Cross-suite differences therefore reflect the full task design, including both external-state grounding and validation density.

APR exhibits the same suite ordering after conditioning on a passing previous turn: 71.1% for Presentation, 68.7% for Interaction, and 52.4% for Tool-grounded tasks (Table 2). A fixed-seed post hoc audit identifies 110 failed APR transitions for which sufficient attribution evidence is available. Among these transitions, 58 (52.7%) involve regression on prior behavior, sometimes together with failure on the new requirement; the remaining 52 (47.3%) preserve prior behavior but fail to satisfy the new requirement (Table 5).

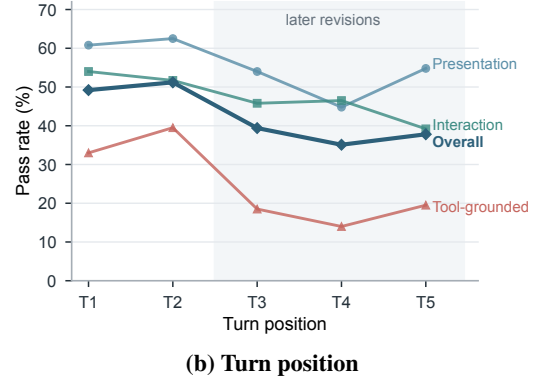
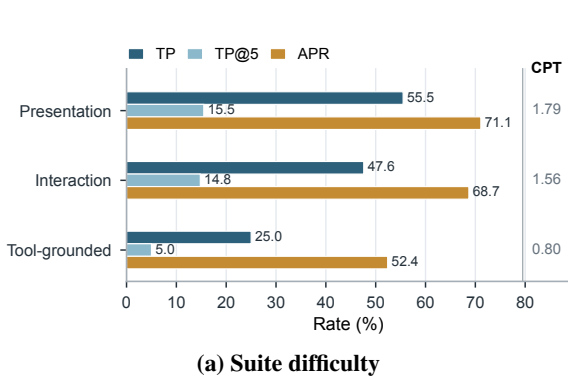


Figure 3: **Main empirical views.** (a) Turn Pass, TP@5, APR, and CPT by suite. (b) Pass rate over requested model–turn slots, with later revisions shaded.

Finding 3: Reliability declines after the second turn.

Aggregate pass rates are similar at the first two turn positions but fall to 39.4% at turn 3 and 35.1% at turn 4 (Figure 3b). The largest aggregate decline occurs between turns 2 and 3. This decline is especially pronounced for Tool-grounded tasks, whose pass rate falls from 39.5% at turn 2 to 18.5% at turn 3 and reaches 14.0% at turn 4. These results indicate that difficulty increases once models must incorporate multiple revisions while maintaining prior interface behavior and state consistency.

5.4 Robustness and Evaluator Diagnostics

The main experiment uses one generation per model–task–turn slot and MiMo-V2.5 as both the interaction actor and evaluator. We therefore conduct additional studies of actor–evaluator sensitivity, evaluator evidence, and regeneration stability. Appendix D.2 provides the complete protocols and results.

Actor–evaluator sensitivity. Across 120 stratified cases, agreement with human annotations ranges from 86.1% to 91.4% across the tested actor–evaluator pairings. The default MiMo–MiMo configuration achieves 89.7% agreement (Table 7), indicating that its performance is within the range of the alternative configurations.

Evaluator evidence ablation. On a separate balanced reference set of 240 cases, the full-evidence evaluator achieves 87.5% accuracy against majority-vote human labels. Removing the actor trace produces the largest decrease, reducing accuracy to 55.0%. Accuracy decreases to 63.8% without the private evaluation reference, 67.5% without the source code, and 78.3% without the screenshot (Table 3). These results indicate that

Configuration	Acc.	Δ Acc.	Prec.	Rec.	F1
Full evidence	87.5	–	86.9	88.3	87.6
No source code	67.5	–20.0	67.2	68.3	67.8
No screenshot	78.3	–9.2	78.3	78.3	78.3
No actor trace	55.0	–32.5	54.7	58.3	56.5
No private reference	63.8	–23.7	92.3	30.0	45.3

Table 3: **Evaluator evidence ablation.** Decision accuracy on a fixed set of 240 balanced cases (120 passing, 120 failing; passing is positive). Δ Acc. is the percentage-point change from full evidence. Each row removes one source.

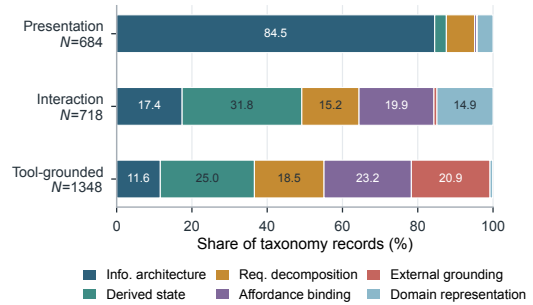


Figure 4: **Diagnostic failure mechanisms.** Values are percentages over all executed non-passing calls in each suite; N denotes the number of calls. Cross-turn preservation is measured separately by APR and TP@5.

no single artifact view is sufficient and that interaction traces are particularly important for identifying behavioral failures. Appendix D.2.1 provides the complete ablation protocol.

Regeneration stability. Table 8 reports three independent runs of Qwen3.6-Plus. Overall TP ranges from 39.7% to 46.4%, while the ordering Presentation (>) Interaction (>) Tool-grounded remains consistent across all three runs. Thus, absolute performance varies across generations, but the principal suite-level difficulty pattern is stable.

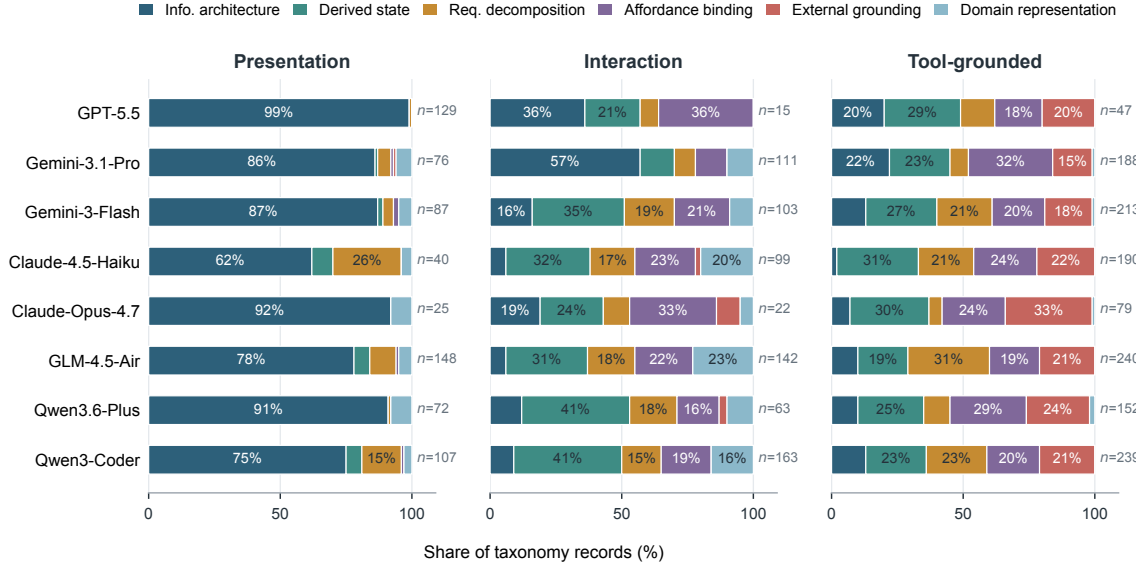


Figure 5: **Failure composition by model and suite.** Each bar is the within-row distribution over six diagnostic mechanisms; n gives the number of executed non-passing calls in that row.

6 Analysis and Discussion

6.1 Diagnostic Failure Taxonomy

We classify each failed execution into one of six interface-maintenance mechanisms: *information architecture*, for poorly organized or unreadable content; *domain representation*, for incorrect task-specific abstractions; *requirement decomposition*, for omitted constraints or workflow steps; *affordance binding*, for visible but unwired controls; *derived-state propagation*, for dependent views that remain stale after local state changes; and *external-state grounding*, for contradictions between the interface and tool or runtime state. The analysis covers all 2,750 executed non-passing calls, each assigned to one primary mechanism: 859 information-architecture, 586 derived-state-propagation, 460 affordance-binding, 410 requirement-decomposition, 289 external-state-grounding, and 146 domain-representation failures. Episode-level reliability and cross-turn retention are analyzed using TP@5 and APR, respectively.

Before large-scale labeling, we constructed and froze the six-label codebook using the evaluation rubrics, private validation criteria, and representative failures. On a category-balanced sample of 120 classified failures (20 per category), a separate diagnostic judge achieves 90.3% mean exact-label agreement with three blinded non-author annotators. Mean multiclass Cohen’s κ between the judge and individual annotators is 0.89, and Fleiss’ κ among the human annotators is 0.91. Ap-

pendix D.2.2 provides the complete labeling and validation protocol.

Figure 4 shows that the dominant bottleneck varies by scenario. Presentation failures primarily involve *information architecture*: requested facts lack a clear hierarchy, overlap visually, or appear clipped. Interaction failures more often involve *derived-state propagation* and *affordance binding*, producing stale dependent views or controls that appear functional but are not wired to behavior. Tool-grounded failures span these mechanisms as well as *external-state grounding* and *requirement decomposition*. Reliable tool-grounded EVOGENUI therefore requires coordinated management of interface and runtime state, not merely valid API invocation. The model-level results in Figure 5 show that these suite-level patterns recur across model families rather than being driven by a single model. Section 6.2 traces three mechanisms through representative artifacts.

6.2 Case Studies

We examine three executed turns from Claude-Opus-4.7, the strongest model overall, to reveal how a plausible interface can conceal a broken mechanism.

Presentation: Carbon Abatement Roadmap Board, Turn 2. The model preserves all six requested wedges, source markers S1–S6, the timeline, and the decision callouts. Execution and Alignment both score 5, but Presentation scores

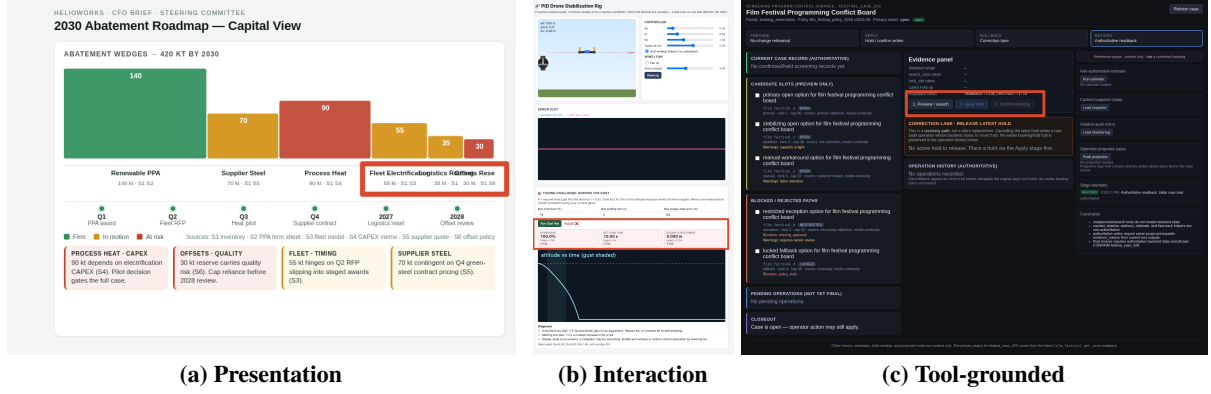


Figure 6: **Three diagnostic failures.** Red boxes mark (a) merged chart labels, (b) metrics that remain stale after gain changes, and (c) controls blocked by mismatched backend evidence tokens.

3 because the SVG chart merges several x-axis labels into one unreadable string. The requested information is present, yet its layout makes it unusable. The screenshot exposes this *information architecture* failure (Figure 6a).

Interaction: Drone PID Wind Disturbance Lab, Turn 5. The generated challenge panel contains target inputs, a *Run Gust Test* button, pass/fail badges, and diagnosis text, earning Presentation 5 and Alignment 4. The actor changes K_p from 2.20 to 1.0, 1.5, and 0.5, but every run returns 100.0% overshoot, 10.00 s settling, and 5.000 m SSE. Although the displayed gain updates, the dependent simulation metrics remain fixed, resulting in an Execution score of 3. The actor trace and source difference expose this *derived-state propagation* failure (Figure 6b).

Tool-grounded: Film Festival Programming Conflict Board, Turn 4. The interface renders the requested correction lane, action rail, and evidence tokens, earning Presentation 5 and Alignment 4. Its tool-calling code, however, sends stale or mismatched prerequisite tokens to the backend, causing every `hold_slot` call to return `missing_prerequisite_token_slots`. After 17 actor steps across multiple interaction paths, no hold succeeds and the correction lane remains unreachable. Execution therefore scores 3. The runtime logs reveal the mismatch between interface state and backend state, identifying an *external-state grounding* failure (Figure 6c).

These cases illustrate different failures require different evidence: the presentation defect is visible in the screenshot, the stale derived state emerges from the actor trace and source difference, and the tool-state mismatch appears in runtime logs.

6.3 Implications for Generative UI Evaluation

The results distinguish two aspects of reliability: whether a model reaches a correct artifact and whether it preserves correctness across subsequent revisions. They also show that reliable diagnosis requires matching evidence to mechanism. Screenshots reveal visual and organizational defects; actor traces and source changes reveal broken interactions and stale derived state; and runtime logs reveal inconsistencies between the interface and external systems. No single score or evidence surface captures all three. Generative UI evaluation should therefore combine outcome-level reliability metrics with mechanism-level diagnosis.

7 Conclusion

We introduced EVOGENUI-BENCH, a benchmark that evaluates generative UI as multi-turn maintenance of an executable artifact. By combining browser execution with visual, behavioral, source-level, and runtime evidence, it measures both turn-level correctness and reliability across revisions. Experiments across model families show that strong single-turn performance does not reliably translate into sustained episode success, particularly for stateful and tool-grounded interfaces.

Our diagnostic analysis further shows that failure mechanisms vary by task, spanning information architecture, interaction wiring, derived-state propagation, requirement decomposition, and external-state grounding. These findings demonstrate that neither a final screenshot nor an isolated turn score adequately captures generative UI reliability. Evaluation must instead assess whether interface behavior, dependent state, external state, and assistant claims remain synchronized as the artifact evolves.

Limitations

EVOGENUI-BENCH uses human-authored five-turn episodes in a fixed React/Vite browser environment. Its scope excludes naturally occurring interaction logs, other UI frameworks, accessibility requirements, and device conditions.

APR is an outcome-level retention measure rather than a causal attribution metric. A failed transition may reflect regression on prior behavior, failure to satisfy the new request, or both. Our post hoc audit distinguishes prior-regression-containing from new-requirement-only failures on a sample, but comprehensive attribution would require replaying all prior-turn validation contracts against each later artifact.

Tool-grounded tasks use deterministic mock runtimes to ensure reproducibility. They therefore evaluate controlled interface-runtime synchronization but do not capture operational challenges of live services, including latency, authentication and permission failures, outages, rate limits, and API changes.

Ethical Considerations

EVOGENUI-BENCH uses synthetic tasks, mock external state, and deterministic tool environments rather than private user data or live third-party services. This reduces privacy risk and supports reproducible evaluation. Before release, we manually screened task prompts, mock fixtures, and public resources for personally identifying information and offensive content. The released artifacts preserve the distinction between generator-visible inputs and private validation material.

Generated interfaces can mislead users when they expose polished but unwired controls, summaries, confirmations, or status labels that are not grounded in executable behavior. EVOGENUI-BENCH probes these risks through evidence-grounded execution, interaction, and tool-state checks. However, it remains a diagnostic benchmark in a controlled setting and does not by itself establish the safety of generated interfaces in deployment.

Acknowledgments

Generative AI tools were used to assist with language polishing and manuscript editing. All generated suggestions were reviewed and verified by the authors, who take full responsibility for the content of the paper.

References

- Alibaba Cloud. 2026. [Qwen3.6-Plus model documentation](#). Alibaba Cloud Model Studio documentation; snapshot qwen3.6-plus-2026-04-02.
- Anthropic. 2025. [Claude Haiku 4.5 system card](#).
- Anthropic. 2026. [Claude Opus 4.7 system card](#).
- Jiaqi Chen, Yanzhe Zhang, Yutong Zhang, Yijia Shao, and Diyi Yang. 2026. [Generative interfaces for language models](#). In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 1499–1519, San Diego, California, United States. Association for Computational Linguistics.
- Kaustubh Deshpande, Ved Sirdeshmukh, Johannes Baptist Mols, Lifeng Jin, Ed-Yeremai Hernandez-Cardona, Dean Lee, Jeremy Kritz, Willow E. Primack, Summer Yue, and Chen Xing. 2025. [Multi-Challenge: A realistic multi-turn conversation evaluation benchmark challenging to frontier LLMs](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 18632–18702, Vienna, Austria. Association for Computational Linguistics.
- Yann Dubois, Percy Liang, and Tatsunori B. Hashimoto. 2024. [Length-controlled AlpacaEval: A simple de-biasing of automatic evaluators](#). In *First Conference on Language Modeling*.
- GLM-4.5 Team. 2025. [GLM-4.5: Agentic, reasoning, and coding \(ARC\) foundation models](#). *Preprint*, arXiv:2508.06471.
- Google DeepMind. 2025. [Gemini 3 Flash model card](#).
- Google DeepMind. 2026. [Gemini 3.1 Pro model card](#).
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. [SWE-bench: Can language models resolve real-world GitHub issues?](#) In *The Twelfth International Conference on Learning Representations*.
- Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. 2024. [Prometheus: Inducing fine-grained evaluation capability in language models](#). In *International Conference on Learning Representations*.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. 2024. [VisualWebArena: Evaluating multi-modal agents on realistic visual web tasks](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905, Bangkok, Thailand. Association for Computational Linguistics.

- Yaniv Leviathan, Dani Valevski, Matan Kalman, Danny Lumen, Eyal Segalis, Eyal Molad, Shlomi Paster-nak, Vishnu Natchu, Valerie Nygaard, Srinivasan Venkatachary, James Manyika, and Yossi Matias. 2026. [Generative UI: LLMs are effective UI generators](#). *Preprint*, arXiv:2604.09577.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2024. [AgentBench: Evaluating LLMs as agents](#). In *International Conference on Learning Representations*.
- Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. 2025a. [ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities](#). In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183, Albuquerque, New Mexico. Association for Computational Linguistics.
- Zimu Lu, Yunqiao Yang, Houxing Ren, Haotian Hou, Han Xiao, Ke Wang, Weikang Shi, Aojun Zhou, Mingjie Zhan, and Hongsheng Li. 2025b. [WebGen-Bench: Evaluating LLMs on generating interactive and functional websites from scratch](#). In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc.
- OpenAI. 2026. [GPT-5.5 system card](#).
- Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Nicholas Roberts, Frederic Sala, and Aws Albarghouti. 2026. [SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks](#). *Preprint*, arXiv:2603.24755.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. [The Berkeley Function Calling Leaderboard \(BFCL\): From tool use to agentic evaluation of large language models](#). In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. [ToolLLM: Facilitating large language models to master 16000+ real-world APIs](#). In *International Conference on Learning Representations*.
- Qwen Team. 2025. [Qwen3-Coder-480B-A35B-Instruct](#). Model card.
- Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2025. [Design2Code: Benchmarking multimodal code generation for automated front-end engineering](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3956–3974, Albuquerque, New Mexico. Association for Computational Linguistics.
- Haoyu Sun, Huichen Will Wang, Jiawei Gu, Linjie Li, and Yu Cheng. 2025. [FullFront: Benchmarking MLLMs across the full front-end engineering workflow](#). *Preprint*, arXiv:2505.17399.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. [AppWorld: A controllable world of apps and people for benchmarking interactive coding agents](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, Bangkok, Thailand. Association for Computational Linguistics.
- Xueqing Wu, Zihan Xue, Da Yin, Shuyan Zhou, Kai-Wei Chang, Nanyun Peng, and Yeming Wen. 2026. [FronTalk: Benchmarking front-end development as conversational code generation with multi-modal feedback](#). In *The Third Annual Conference on Language Modeling*. To appear.
- Jingyu Xiao, Yuxuan Wan, Yintong Huo, Zixin Wang, Xinyi Xu, Wenxuan Wang, Zhiyao Xu, Yuhang Wang, and Michael R. Lyu. 2025. [Interaction2Code: Benchmarking MLLM-based interactive webpage code generation from interactive prototyping](#). In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025)*, pages 241–253.
- Xiaomi MiMo Team. 2026. [MiMo-V2.5](#). Model card.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. [OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 52040–52094. Curran Associates, Inc.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. [\$\tau\$ -bench: A benchmark for Tool-Agent-User interaction in real-world domains](#). In *International Conference on Learning Representations*.
- Chenchen Zhang, Yuhang Li, Can Xu, Jiaheng Liu, Ao Liu, Changzhi Zhou, Ken Deng, Dengpeng Wu, Guanhua Huang, Kejiao Li, Qi Yi, Ruibin Xiong, Shihui Hu, Yue Zhang, Yuhao Jiang, Zenan Xu, Yuanxing Zhang, Wiggin Zhou, Chayse Zhou, and Fengzong Lian. 2025. [ArtifactsBench: Bridging the](#)

visual-interactive gap in LLM code generation evaluation. *Preprint*, arXiv:2507.04952.

Zuhao Zhang, Chengyue Yu, Yuante Li, Chenyi Zhuang, Linjian Mo, and Shuai Li. 2026. [MiniAppBench: Evaluating the shift from text to interactive HTML responses in LLM-powered assistants](#). In *Proceedings of the 43rd International Conference on Machine Learning*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. [Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. [WebArena: A realistic web environment for building autonomous agents](#). In *International Conference on Learning Representations*.

Hongda Zhu, Yiwen Zhang, Bing Zhao, Jingzhe Ding, Siyao Liu, Tong Liu, Dandan Wang, Yanan Liu, and Zhaojian Li. 2025. [FrontendBench: A benchmark for evaluating LLMs on front-end development via automatic evaluation](#). *Preprint*, arXiv:2506.13832.

A Benchmark Artifacts and Release

A.1 Benchmark Card

- **Intended use:** comparing models on multi-turn generative UI construction and revision.
- **Not intended for:** certifying safety or production readiness of generated applications.
- **Primary metrics:** Turn Pass, TP@5, APR, CPT, and dimension scores.
- **Required artifacts:** generated source files, assistant text, build logs, browser snapshots, interaction traces, tool logs when applicable, evaluator outputs, and retention summaries.
- **Release:** Version v1.0 is available under the MIT License at <https://github.com/MAPS-research/EvoGenUI-Bench>; Appendix A.2 gives the release protocol.

A.2 Artifact Release and Licensing

Repository. The complete code, data, and pipeline are available under the MIT License at <https://github.com/MAPS-research/EvoGenUI-Bench>, with a README and reproduction instructions.

Versioning. The benchmark version used in this paper is v1.0. Any reported evaluation result should identify the benchmark version, code commit, task manifest, prompt versions, and evaluator/actor configuration, because changes to tasks, prompts, hidden validation contracts, or runtime fixtures can affect scores.

Licensing. The MIT License covers the code, task data, mock fixtures, public resources, documentation, and benchmark artifacts. Model outputs and interaction traces use the same license where model-provider terms permit it; covered materials are marked in the repository.

Public and evaluator-only components.

Generator-visible materials include user turns, public context, public tool/resource contracts, prompt templates, runtime code, and evaluation scripts. Evaluator-only materials include hidden validation contracts, actor-only facts, backend fixtures, runtime state, and evaluator-specific evidence requirements. Both are released for auditing and reproduction, but the execution pipeline never exposes evaluator-only fields to generator models.

B Task Suite Design

B.1 Task Authoring Workflow

The authoring workflow is:

1. Choose a domain and interface artifact type, such as a planning dashboard, evidence board, simulator, reasoning workbench, map, or action console. The domain is checked against the suite manifest to avoid duplicate domain labels and near-duplicate interface types.
2. Write a five-turn user trajectory in which each turn introduces a cumulative revision to the same artifact rather than an independent mini-app.
3. Specify private validation requirements for each turn, including requirements from earlier turns that remain active.
4. Bind each requirement to one or more evidence surfaces: screenshot, DOM, interaction trace, tool log, external state, source changes, visible UI text, or assistant text.

5. Add task metadata describing the targeted challenge, such as information hierarchy, linked derived-state propagation, state-machine invariants, affordance binding, tool read/write grounding, authoritative readback, or text–artifact calibration.
6. Cross-validate the task with a different human reviewer, who checks ambiguity, domain duplication, rubric leakage, observability of the validation contract, and whether later turns genuinely require multi-turn maintenance rather than independent regeneration.

Exact positions or wording are enforced only when the user requests them; otherwise, semantically equivalent layouts and interaction paths remain valid.

B.2 Common Task Schema

Each task in EVOGENUI-BENCH is a five-turn JSON episode. The abridged structure below retains the fields that define generator-visible input and private validation; curation metadata is omitted.

Shared Task Structure (abridged)

```
{
  "task_id": "...",
  "domain": "...",
  "suite": "presentation_ui | interactive_tool_ui |
    tool_grounded_action_ui",
  "core_interaction": ["..."],

  "metadata": {
    "task_world": {
      "public_context": "..."
    },
    "validation_contract": {
      "validation_scenarios": [
        {
          "turn": 1,
          "evidence_requirements": [
            {
              "surface": "screenshot / DOM / actor trace /
                tool log / runtime state / source / assistant text",
              "expect": "..."
            }
          ]
        }
      ]
    }
  },

  "turns": [
    {
      "turn": 1,
      "prompt": "..."
    }
  ]
}
```

The generator receives the public task context and turn prompts. The validation contract remains private and binds expected behavior to one or more observable sources. `core_interaction` records

the mechanisms stressed by the task for suite-level analysis.

Tool-grounded tasks add public tool contracts plus private bindings and backend state.

Tool-Grounded Extension (abridged)

```
{
  "tools": [
    {
      "name": "...",
      "mode": "read | write",
      "mock_contract": {
        "fixture_id": "...",
        "handler_ref": "...",
        "semantic_contract": {}
      }
    }
  ],

  "initial_state": {}
}
```

The model sees each tool’s name, description, and public schema. Fixture bindings, initial state, and validation requirements remain private to execution and evaluation.

B.3 Representative Task Example

Pharmacy Prior Authorization Control Surface asks the model to maintain an appeal console for case `rx_case_592`. Its five revisions progress from staging and no-write rehearsal to guarded mutation, readback, recovery, and audit closeout. The table-and-detail workspace separates current state, candidate actions, blocked alternatives, rehearsal output, pending work, and final readiness. The trajectory tests grounded read/write behavior, resistance to non-authoritative helpers, and audit-history preservation.

The hidden contract checks observable evidence rather than a reference implementation: family-tool use, backend-derived selected and blocked records, post-write readback, a visible interaction path, and assistant text consistent with backend state. Later turns add no-write rehearsal, scoped writes, reversible recovery, and evidence-chain reconstruction.

C Evaluation Protocol and Metrics

C.1 Evaluation Rubrics

The rubric expands the three evaluator-scored dimensions in Section 4.2. Each uses a 1–5 scale and passes at scores of 4 or 5. Appendix C.2 defines the statistical metrics derived from these turn-level decisions.

Tool group	Examples
Authoritative read tools	get_case, search_records, get_record_details, compare_records, get_audit_log
Authoritative write tool	save_selection
Decoy or non-authoritative helpers	cached snapshot, estimate, advisory check, fast-track request, shadow audit log, optimistic status, stale evidence token, draft receipt, commit-phrase validation
Near-miss same-domain tools	entity resolution, profile lookup, capacity pools, simulated allocation, slot search, policy note

Table 4: **Tool groups.** Authoritative, decoy, and near-miss tools in the pharmacy task.

Presentation. Presentation assesses the rendered UI quality: layout, hierarchy, readability, styling, discoverability, and whether screenshot evidence shows a coherent domain-appropriate interface rather than a sparse scaffold, wireframe, or visibly broken page. For screenshot-grounded artifacts, DOM labels and source code can show that objects exist, but they cannot prove that labels are readable, unclipped, or visually coherent.

Execution. Execution assesses whether the current turn’s requested requirements are implemented as working UI behavior or visible, evidence-backed state. It covers required surfaces, interaction wiring, state updates, tool/resource calls, derived outputs, committed readback, confirmations, and relevant still-valid prior requirements. A button, form, raw tool response, or optimistic confirmation does not pass if the requested result or dependent surface is missing, stale, disconnected, or unsupported by runtime evidence.

Alignment. Alignment assesses whether assistant text, generated source, visible UI, interaction observations, and runtime logs describe the same concrete behavior. It penalizes unsupported claims, stale descriptions, claims of unavailable capabilities, claimed saved/submitted/filtered/computed state without evidence, and mismatches between the UI state and the assistant’s explanation.

C.2 Evaluation Metrics

Section 4.3 defines the official turn-, episode-, and transition-level metrics. Countable-only variants are diagnostic rather than official: they remove provider failures for which no model response was obtained. Interaction-agent status and static code analysis are retained for evidence and attribution rather than used as direct pass gates.

Task-level bootstrap 95% confidence intervals resample complete episodes with replacement, preserving within-episode correlations. Table 9 reports percentile intervals and the TP@5 independence

baselines.

C.2.1 Post Hoc Attribution of Failed APR Transitions

We audit a fixed-seed sample of eligible transitions in which the previous turn passed and the following turn failed. For each transition, annotators inspect the previous user request, previous passing evidence, current user request, current failure evidence, generated source and DOM evidence, screenshots, browser interaction traces, and tool/runtime logs when available. The audit is diagnostic only and does not affect the official APR calculation.

We assign each attributable failed transition to one of two categories. *Prior-regression-containing* means that at least one behavior, state, surface, or still-valid requirement supported by the previous passing artifact was removed, contradicted, disconnected, or made stale in the next revision. The same transition may also contain a failure on the newly introduced requirement. *New-requirement-only* means that the previous behavior remains largely intact and the failed transition is primarily explained by the current turn’s newly introduced or revised requirement.

Attribution category	Count	Share
Prior-regression-containing	58	52.7%
New-requirement-only	52	47.3%

Table 5: **Failed APR transition audit.** Post hoc attribution of $N = 110$ failed APR transitions with sufficient evidence.

D Experimental Configuration and Validation

D.1 Evaluated Models

Table 6 lists the generator models and inference settings used for the final reported evaluation slice. Once the final run set was selected, completed

Model Alias	Exact Model Identifier	Temp.	Output Token Cap
GPT-5.5	gpt-5.5	0	32,768
Gemini-3.1-Pro	gemini-3.1-pro-preview	0	32,768
Gemini-3-Flash	gemini-3-flash-preview	0	32,768
Claude-4.5-Haiku	claude-haiku-4-5-20251001	0	32,768
Claude-Opus-4.7	claude-opus-4-7	0	32,768
GLM-4.5-Air	glm-4.5-air	0	32,768
Qwen3.6-Plus	qwen3.6-plus	0	32,768
Qwen3-Coder	qwen3-coder-480b-a35b-instruct	0	32,768

Requested-slot accounting		
Slot category	Count	Share
Executed generation calls	5,310	88.5%
Blocked after earlier build failure	443	7.4%
Blocked after earlier invalid output	247	4.1%
Total requested slots	6,000	100.0%

Table 6: **Evaluated models and requested-slot accounting.** The upper table lists generator identifiers and inference settings; output token cap is the configured completion limit, not observed mean length. The lower table separates executed calls from unexecuted downstream slots. Unexecuted slots remain failures for Turn Pass, TP@5, and CPT.

Check	Actor	Evaluator	Interaction	Presentation	Tool-grounded	Overall
Default	MiMo	MiMo	90.8%	88.3%	90.0%	89.7%
Fixed actor	MiMo	Qwen3.6-Plus	88.3%	85.8%	87.5%	87.2%
Fixed actor	MiMo	GPT-5.5	91.7%	89.2%	90.8%	90.6%
Fixed evaluator	Qwen3.6-Plus	MiMo	89.2%	86.7%	88.3%	88.1%
Fixed evaluator	GPT-5.5	MiMo	91.7%	88.3%	90.0%	90.0%
Same model	Qwen3.6-Plus	Qwen3.6-Plus	87.5%	84.2%	86.7%	86.1%
Same model	GPT-5.5	GPT-5.5	92.5%	90.0%	91.7%	91.4%

Table 7: **Actor–evaluator sensitivity.** Diagnostics on $N = 120$ cases (40 per suite). MiMo denotes MiMo-V2.5. Each case receives three human annotations; suite and overall percentages therefore aggregate 120 and 360 individual annotations per configuration, respectively. Human Agreement is the percentage of those annotations that support the automatic decision. Bold entries exceed the default MiMo–MiMo configuration within the same suite.

model outputs were frozen for downstream analysis.

All settings refer to generator calls; the token value is a completion cap, not mean output length. The default MiMo-V2.5 actor and evaluator use temperature 0, output caps of 4,096 and 8,192 tokens, respectively, and fixed prompts across generator models.

D.2 Robustness Diagnostics

D.2.1 Evaluator Evidence and Component Ablation

The ablation uses a fixed reference set of 240 frozen artifact packets, balanced between 120 passing and 120 failing cases. Each packet includes the user request, generated UI, assistant response, build result, browser trace, final DOM, screenshot, and runtime logs.

We rerun the evaluator on each frozen artifact with full evidence or with one source removed: generated code, screenshot, actor trace, or private evaluation reference. Removing the actor trace leaves the final DOM, screenshot, and runtime logs; removing the private reference leaves public task

context and observable artifacts. Metrics compare the automatic turn-level decision with the fixed reference label, treating passing turns as the positive class.

D.2.2 Failure Taxonomy Construction and Validation

Before large-scale labeling, we grouped recurring errors from the evaluation rubrics, private validation criteria, and manually inspected failures into six categories, merged overlaps, and froze the codebook.

A diagnostic protocol separate from the turn-level evaluator assigns every executed non-passing call to one of the six mechanisms using current- and previous-turn context, screenshots, source/DOM evidence, actor/tool traces, and build or output evidence where applicable. These labels are diagnostic and do not affect official pass/fail metrics. Three non-author annotators, blinded to judge labels and model identities, independently label a category-balanced sample of 120 failures (20 per category) to evaluate agreement with the diagnostic judge.

Suite	Run	Turn Pass	Pres.	Exec.	Align.	Avg. Score
Presentation	R1	60.0%	3.86	3.71	3.82	3.80
	R2	55.2%	3.74	3.58	3.69	3.67
	R3	62.0%	3.93	3.78	3.88	3.86
	Mean $\pm$ SD	59.1 $\pm$ 3.5	3.84 $\pm$ 0.10	3.69 $\pm$ 0.10	3.80 $\pm$ 0.10	3.78 $\pm$ 0.10
Interaction	R1	46.8%	3.62	3.28	3.55	3.48
	R2	42.8%	3.48	3.12	3.41	3.34
	R3	49.2%	3.70	3.37	3.63	3.57
	Mean $\pm$ SD	46.3 $\pm$ 3.2	3.60 $\pm$ 0.11	3.26 $\pm$ 0.13	3.53 $\pm$ 0.11	3.46 $\pm$ 0.12
Tool-grounded	R1	25.2%	3.31	2.91	3.12	3.11
	R2	21.2%	3.17	2.72	2.95	2.95
	R3	28.0%	3.42	3.05	3.25	3.24
	Mean $\pm$ SD	24.8 $\pm$ 3.4	3.30 $\pm$ 0.13	2.89 $\pm$ 0.17	3.11 $\pm$ 0.15	3.10 $\pm$ 0.15
Overall	R1	44.0%	3.60	3.30	3.50	3.46
	R2	39.7%	3.46	3.14	3.35	3.32
	R3	46.4%	3.68	3.40	3.59	3.56
	Mean $\pm$ SD	43.4 $\pm$ 3.4	3.58 $\pm$ 0.11	3.28 $\pm$ 0.13	3.48 $\pm$ 0.12	3.45 $\pm$ 0.12

Table 8: **Regeneration stability.** Three independent Qwen3.6-Plus generations on the full 150-task, 750-turn slice. Pres., Exec., and Align. are mean 1–5 evaluator scores; Avg. Score averages the three dimensions.

Model	Turn Pass [95% CI]	TP@5 [95% CI]	CPT [95% CI]	APR [95% CI]	Indep. TP@5
GPT-5.5	61.7% [56.9, 66.1]	21.3% [14.7, 28.0]	2.21 [1.89, 2.51]	71.0% [65.7, 75.8]	8.7%
Qwen3.6-Plus	44.0% [38.5, 48.4]	7.3% [3.3, 11.3]	1.32 [1.03, 1.57]	61.7% [55.4, 67.5]	1.3%
Gemini-3-Flash	41.7% [36.8, 46.7]	4.7% [1.3, 8.0]	1.02 [0.78, 1.27]	57.8% [51.7, 63.8]	1.2%
Claude-Opus-4.7	74.9% [70.8, 79.6]	37.3% [30.0, 45.3]	2.81 [2.51, 3.14]	83.6% [79.9, 86.9]	23.4%
Qwen3-Coder	23.2% [18.8, 27.9]	2.0% [0.0, 4.7]	0.77 [0.58, 0.97]	42.2% [33.6, 50.5]	<0.1%
GLM-4.5-Air	21.1% [16.7, 25.2]	1.3% [0.0, 3.3]	0.60 [0.41, 0.77]	48.5% [39.7, 56.1]	<0.1%
Gemini-3.1-Pro	23.6% [19.1, 28.3]	6.7% [3.3, 11.3]	0.69 [0.49, 0.91]	68.6% [60.8, 75.4]	0.1%
Claude-4.5-Haiku	51.1% [45.9, 56.5]	13.3% [8.7, 18.7]	1.61 [1.31, 1.92]	65.7% [59.4, 70.9]	3.2%

Table 9: **Metric uncertainty.** Task-level bootstrap 95% confidence intervals for the four aggregate metrics, with the independence baseline for TP@5 (Indep. TP@5). TP, TP@5, and APR are percentages; CPT is in turns out of five. All rows use task-level episode resampling under the denominator conventions in Appendix C.2.

E Qualitative Analysis and Visualization Tools

E.1 Web-based Visualization Interface

EVOGENUI-BENCH includes a local interface for inspecting benchmark composition and run artifacts.

The results view connects aggregate scores to the evidence for a selected turn. A manual audit can inspect the request, source, screenshot, actor trace, tool logs, backend state, and evaluator output in one record.

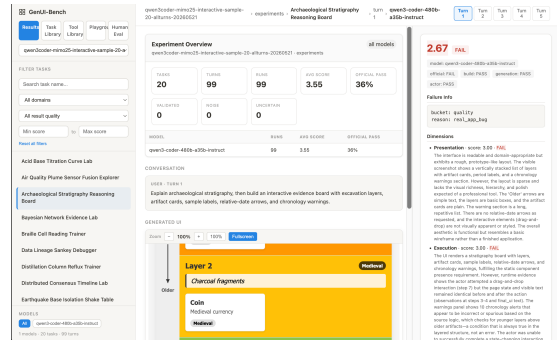


Figure 8: **Experiment results.** Aggregate scores link to the generated UI, execution status, actor result, and dimension-level evaluator feedback.

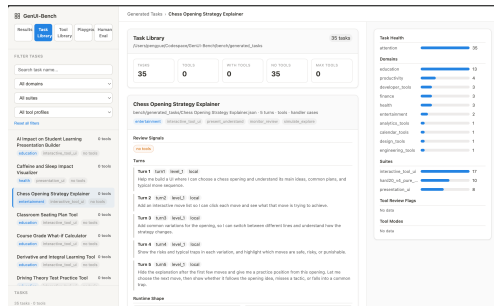


Figure 7: **Task library.** The web interface summarizes task metadata, domain distribution, suite membership, tool availability, and user turns.

F Prompt Templates

F.1 Generator Prompt

Every model receives the same system prompt.

Generator prompt excerpt

You are an LLM chatbot with the ability to build a small, self-contained React app in a sandboxed workspace when it is helpful. Output assistant_text: followed by a non-empty user-facing response, then complete source file sections. Treat the app as one accumulated ar-

tifact across turns. Implement the current request while preserving prior behavior that is still relevant. For tool-backed workflows, use only declared `callTool` and `readResource` capabilities, show returned results and committed readback, and do not fake tool results.

F.2 Turn Payload Template

At turn t , the payload combines the current request and public task contracts with compact prior-turn context and the latest source files.

Turn Payload Template

```
{
  "current_user_request": "...",
  "task": {
    "context": "...",
    "tools": [... public contracts ...],
    "resources": [... public contracts ...],
    "conversation_history": [...]
  },
  "previous_turns": [
    {
      "turn": 1,
      "user_request": "...",
      "assistant_text": "...",
      "final_ui_text": "..."
    }
  ],
  "previous_turn_source": {
    "turn": 1,
    "files": {
      "src/App.tsx": "...",
      "src/App.css": "..."
    }
  }
}
```

F.3 Browser Actor Prompt

Browser actor prompt excerpt

Open the generated app and evaluate whether it satisfies the current user request. Prioritize the current turn's request over exhaustive testing. Identify the primary action, execute it, inspect the result, and finish once decisive evidence is available. Runtime logs show which tool calls actually happened; use them to verify behavior.

F.4 Evaluator Prompt

Evaluator prompt excerpt

You are the EVOGENUI-BENCH evaluator. Grade generated UI code based strictly on observable evidence. Use only the supplied evidence. Source code is not runtime proof. Return exactly one JSON object with one top-level key per requested dimension; each value must contain score, summary, and failure_types.